A posterior's training cost is memory traffic over parameter-sized buffers

Tage Sköldefors

August 2026

Abstract

What a Bayesian posterior costs per training step, on a GPU, is the bytes its optimiser moves over parameter-sized buffers, and that quantity does not depend on how many tokens the step processes. Bayes-by-Backprop, Vadam, a learned low-rank-plus-diagonal Gaussian covariance and a five-member deep ensemble are implemented as four optimisation strategies over one CUDA GPT-2 of 11,061,531 parameters, sharing kernels, batches, optimiser constants and precision, so an arm differs only in what it stores and how many passes it makes over that store. Timed region by region with CUDA events across a 256-fold batch sweep at fixed sequence length, each arm's posterior work holds to within 1.5% of itself while tokens per step go from 64 to 16,384. Its share of the step therefore falls towards $1/B$, at a local exponent of 1.005 for Bayes-by-Backprop at $B = 256$, but not below $B \approx 16$, where the network's own forward and backward pass is launch-bound rather than token-bound: $1/B$ is the asymptote and not the curve. Folding each arm's parameter-sized passes into one kernel cuts Bayes-by-Backprop's $O(P)$ time by 1.76× and leaves Vadam's inside measurement drift, which its pass count predicts: 11 buffer touches per step against a floor of 9. Evaluation at $S = 32$ is byte-identical across that change on all four arms, and the fused kernels are bit-identical to the ones they replace with multiply-add contraction disabled.

1 Introduction

A step of Bayesian training on a GPU costs what its optimiser moves. Every parameter-sized kernel here runs at the card's memory bandwidth, so a posterior's per-step price reduces to two counts: how many parameter-sized buffers the method keeps, and how many times a step reads or writes each. Neither count contains the batch size.

Four optimisation strategies are implemented over one network to make those counts measurable. Bayes-by-Backprop [1], Vadam [11], a learned low-rank-plus-diagonal Gaussian covariance and a five-member deep ensemble [14] share a single CUDA implementation of a GPT-2 [20]: the same kernels, the same forward and backward pass, the same batches in the same order, the same AdamW hyperparameters, the same precision. Each arm's source file holds only what it stores and how it updates it, and the ensemble stores no posterior, so every buffer the other three add is attributable to their optimiser. Every region of every step is bracketed by CUDA events at the launch sites, so each arm's Bayesian work is timed rather than inferred as a difference against a baseline that also differs elsewhere.

2 Related work

Weight uncertainty was priced as description length by Hinton and van Camp [7], made practical for real networks by Graves [5], and turned into Bayes-by-Backprop by Blundell et al. [1] through the reparameterisation of Kingma and Welling [13], which pays for a posterior with a second parameter-sized tensor and a second optimiser state; Wen et al. [25] move a per-example perturbation into the activations. Khan et al. [11] instead reuse Adam's second moment as a posterior precision and allocate nothing beyond it, Osawa et al. [18] scale natural-gradient variational inference to ImageNet, and Shen et al. [23] apply it at GPT-2 scale. Covariances fitted after training move the cost out of the step: a Kronecker-factored Laplace approximation at a trained point [21], the same approximation as a recipe [3], and a low-rank-plus-diagonal factor from the SGD trajectory [16], which is the covariance the third arm here learns during training instead. Deep ensembles [14] keep the step deterministic and pay E training runs, and match or beat variational methods on the benchmarks of Ovadia et al. [19]; MC dropout [4] is the other cheap baseline, and this network has no dropout layers. Metrics follow Guo et al. [6] and Kendall and Gal [10].

3 Four optimisation strategies over one network

The shared header holds the kernels, buffer layout, forward and backward pass, dataloader, checkpoint format and evaluation harness; each arm's source file holds only its optimiser. Cost is counted there in *touches*, one read or one write of a parameter-sized buffer in a step, which at $P = 11{,}061{,}531$ parameters in fp32 is 44.2 MB. A deterministic AdamW step costs eight: a gradient zeroed and read, weights and two moments each read and written.

Each arm is counted twice below, as the passes the method needs and as the passes left after the redundant ones are folded into one kernel. One observation does most of that folding in all three Bayesian arms: a gradient accumulator for a variational parameter is write-only, so it never has to reach memory and becomes a register, and the next step's draw folds in with it as a function of the parameters the update just wrote. The fold is gated as an implementation of the same arithmetic (Section 6).

Bayes-by-Backprop. A fully factorised Gaussian, drawn as $w_i = \mu_i + \mathrm{softplus}(\rho_i)\,\varepsilon_i$ once per step for the whole batch and reparameterised so the gradient reaches both moments [13, 1], against a prior of standard deviation 0.1 under an ELBO whose KL weight is $1/N$ for N the training token count, read off the loader rather than tuned. It stores μ, ρ and an AdamW moment pair for each, where a deterministic model stores weights and one pair, and runs AdamW twice a step: eight launches and 32 touches over ten buffers, the most of the four. Folding leaves one launch and 15 touches over eight, the floor its state implies.

Vadam. Adam's second moment already estimates a curvature, so Khan et al. [11] reuse it as the posterior precision and allocate nothing for the posterior. The mean alone is free and the scale is read off the optimiser,

$$\sigma_i = \frac{1}{\sqrt{N s_i + \lambda}}, \qquad w_i = \mu_i + \sigma_i \varepsilon_i, \tag{1}$$

with $\lambda = 1/\sigma_{\text{init}}^2 = 2500$ the prior precision, the initialisation rather than a tuned value, and the square root absent from σ following that paper's footnote 2. Its update is Figure 1 of the same paper transcribed unchanged, one Adam step with the prior folded into the gradient, so the state is

μ with Adam's own m and s: three launches and 11 touches against a floor of 9, two touches of slack and no accumulator to delete.

Low-rank plus diagonal covariance. This arm buys correlation with memory. The covariance is $\Sigma = AA^\top + \mathrm{diag}(d^2)$ at rank $r = 8$, never formed, and a draw is $w_i = \mu_i + \sum_{k<r} A_{ik} z_k + d_i \varepsilon_i$ with one $z \sim \mathcal{N}(0, I_r)$ per weight matrix per forward pass, shared by all P of its weights; that shared draw is the coupling, and the KL reaches $\log\det\Sigma$ through an $r \times r$ determinant. The posterior covers the four GEMM weight matrices per block, 24 tensors holding 10,616,832 parameters. Counted per Bayesian layer in units of that layer's parameter count, so that A counts as r, it costs two launches per layer plus four global, 122 units at $r = 8$, folding to one launch per layer and 58. It is the one arm left above its floor: its KL setup goes through a cuBLAS call per layer and a Cholesky on the host, so it cannot fold into the update it feeds, needs a reduction over each layer rather than one index, and round-trips $r \times r$ matrices through the host.

Deep ensemble (control). $E = 5$ deterministic models [14] differing only in the initialisation seed, which with no dropout and a sequential loader is the only source of diversity; member probabilities are averaged at inference, never logits. The arm stores no posterior and introduces no kernels, so its step is the deterministic two launches and eight touches, already at the floor. It is left unfused on purpose: its in-training output is the reference the repository is calibrated against and its code is identical in both sweeps, which makes it the drift control in Section 5. Its cost sits elsewhere, at E times the step, E times the memory, E training runs, and an evaluation-time sample count capped at the members it paid to train.

4 Setup and measurement protocol

Model, data, hardware. A GPT-2 [20] of the usual pre-LayerNorm shape [24] at $C = 384$, 6 heads, 6 blocks, $V = 27$, trained at $T = 64$ and $B = 4$: 11,061,531 parameters, the six blocks holding about 96%. Attention is the IO-aware formulation of Dao et al. [2]; format, loader and reference implementation follow llm.c [9] and nanoGPT [8]. The corpus is text8 [17], character-level over 27 symbols in the standard 90M/5M/5M split, walked sequentially so every arm sees the same tokens in the same order, for 100,000 steps. One RTX 2060 Max-Q, `sm_75`, 6 GB, fp32 throughout, the four GEMMs per block through cuBLAS and every other kernel checked against a single-threaded C reference. AdamW [12, 15] everywhere at learning rate 10^{-4}, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, no weight decay, no schedule, and Philox 4x32-10 [22] for randomness, so a perturbation is a pure function of (seed, step, weight index) and is regenerated in the backward pass rather than stored.

Timing a region. Step times come from the real training step with CUDA event marks at region boundaries, one region per kernel group, rather than from a copy of it that can drift, and a region's name records whether its cost is $O(P)$ and Bayesian, $O(P)$ and paid by a deterministic trainer too, or $O(B \cdot T)$. Warm-up is discarded, 30 steps at $B \le 8$ falling to 10 at $B = 256$, and the statistic is a median over the 40 to 270 retained steps with $p25$ and $p75$ beside it, so a throttled bimodal sample stays visible instead of smearing into a mean. Temperature and SM clock are probed every 20 steps and printed with each measurement, the binary aborts above 89 °C, and the driver idles between points until the card is under 72 °C. A point with less than 384 MB of VRAM headroom is reported out of range rather than measured (Section 8). The sweep runs $B = 1$ to 256 at $T = 64$ held. Touch counts are cross-checked rather than trusted: converting a region's time to achieved bandwidth (Table 1) has to put structurally unrelated regions in one band, and a wrong count lands off it.

Region	Touches	ms at $B = 4$	GB/s	ms at $B = 128$	GB/s
`*sample`	3	TODO	TODO	TODO	TODO
`+zero_dw`	1	TODO	TODO	TODO	TODO
`*zero_post`	2	TODO	TODO	TODO	TODO
`*post_bwd`	6	TODO	TODO	TODO	TODO
`*kl`	6	TODO	TODO	TODO	TODO
`+adamw_mu`	7	TODO	TODO	TODO	TODO
`*adamw_rho`	7	TODO	TODO	TODO	TODO
all $O(P)$ work	32	TODO	TODO	TODO	TODO
`*fused_step`	15	TODO	TODO	TODO	TODO

Table 1: Every parameter-sized region of the mean-field step, touches counted at the launch sites and converted at $P \cdot 4$ bytes each. The aggregate row is the sum of the rows above it; `*fused_step` is the same work after folding.

Measuring quality. Evaluation loads a checkpoint and never trains, and each arm supplies only a callback returning draw s. Over S draws per position, with predictive distributions p_s and mixture $\bar{p} = \frac{1}{S}\sum_s p_s$, the shared harness reports negative log predictive density of mixture and single draw, expected calibration error over 15 bins of argmax confidence [6], and the decomposition [10]

$$\underbrace{H[\bar{p}]}_{\text{total}} = \underbrace{\tfrac{1}{S}\textstyle\sum_s H[p_s]}_{\text{aleatoric}} + \underbrace{\tfrac{1}{S}\textstyle\sum_s \mathrm{KL}(p_s \,\|\, \bar{p})}_{\text{epistemic}}, \tag{2}$$

in nats, in double precision, mixing samples as probabilities and never as logits. Both halves of the right-hand side are computed directly rather than one from the other, so the identity, asserted at 10^{-5}, tests the accumulation and not the posterior. The off-distribution stream is uniform-random token ids from a fixed seed.

5 What the posterior costs

5.1 The cost is fixed per step

Each arm's posterior work costs the same milliseconds at 64 tokens per step as at 16,384. Across a 256-fold batch sweep the directly timed Bayesian regions hold to within 1.1% of themselves for mean-field, 1.5% for Vadam and 1.4% for the low-rank arm (Table 2): the width of the measurement, not a trend in it.

Nothing in those regions has a token in it. They stream parameter-sized buffers, and how many tokens produced the gradient in one does not change its length. The traffic is what the time is: every parameter-sized region of the mean-field step lands between 234.7 and 253.7 GB/s and their aggregate at 240.8 GB/s (Table 1), so regions sharing no arithmetic differ only in how many buffers each walks.

5.2 Amortisation, and where it stops

The network's cost grows with tokens and the posterior's does not, so the posterior's share of a step falls: mean-field from 34.40% at $B = 1$ to 0.75% at $B = 256$, at a local exponent of 1.005 over the last two points, Vadam at 0.998 (Table 2).

That asymptote is not the whole curve. Between $B = 1$ and $B = 2$ the exponent is 0.148 for mean-field and 0.196 for Vadam, and the reason is the denominator: the network is not $O(B \cdot T)$ at

Arm	B range	Posterior ms (min)	(max)	Spread	Share at $B = 1$	Share at max B	k (small end)	k (top end)
BBB	1–256	4.390	4.437	1.1%	34.40%	0.75%	0.148	1.005
Vadam	1–256	1.844	1.872	1.5%	20.52%	0.31%	0.196	0.998
LowR	1–128	38.470	39.001	1.4%	79.71%	11.65%	0.036	0.824

Table 2: Each arm's own Bayesian work, timed at the launch sites, unfused. Share is the posterior over the whole step, k the exponent in share $\propto B^{-k}$ between the two smallest and the two largest points measured.

B	Tokens/step	BBB ms	BBB $O(P)$%	Vadam ms	Vadam $O(P)$%	LowR ms	LowR $O(P)$%	Ens ms	Ens $O(P)$%
1	64	10.71	31.4	9.16	21.6	42.66	80.8	8.45	17.7
2	128	11.99	28.2	10.42	19.1	44.20	78.7	9.83	15.1
4	256	15.23	22.6	13.77	14.7	47.49	74.1	13.18	11.2
8	512	24.18	14.3	22.69	9.0	56.99	62.2	22.16	6.7
16	1024	43.18	8.1	41.68	4.9	76.04	46.4	41.20	3.6
32	2048	79.27	4.4	77.78	2.6	113.13	31.4	77.70	1.9
64	4096	149.81	2.3	148.59	1.4	182.67	19.2	148.82	1.0
128	8192	295.16	1.2	294.54	0.7	329.50	10.7	292.52	0.5

Table 3: The batch sweep for the shipped code. $O(P)$% is all parameter-sized work over the step, Bayesian or not. The ensemble column is one member, no posterior.

the bottom of the sweep either. The control arm shows that with no posterior in the way, costing 8.45 ms at 64 tokens and 9.83 ms at 128, a factor of 1.16 for twice the work, while from $B = 16$ upward each doubling of tokens doubles the step (Table 3). Below it a forward and backward pass costs more in launches than in tokens, so $1/B$ is the asymptote and not the curve.

The fixed numerator is not a property of posteriors either: the control's own parameter-sized work falls from 17.7% of its step to 0.5% over the same sweep. What fails to amortise is state not proportional to the parameter count, which is why the low-rank arm still owns 11.65% of a step at $B = 128$, at an exponent of 0.824. And all of it is per-step cost, so a sweep over B prices neither the ensemble's E runs nor a variational arm's S passes at evaluation.

5.3 The removable part is the redundant traffic

Folding an arm's parameter-sized passes into one kernel buys time in proportion to the traffic it deletes, and nothing where there is none to delete. Mean-field drops 32 touches to 15 and its $O(P)$ time by 1.76× at $B = 1$, falling to 1.65× at $B = 128$; the low-rank arm drops 122 units per layer to 58 and gains 1.14 to 1.16×; Vadam drops 11 to 9 and lands between 0.98 and 1.04× (Table 4). The control is unchanged code measured in both sweeps and comes out within 1% of itself, so anything inside that band is drift, which is where Vadam's sits.

That null is what Vadam's pass count predicts. Two touches of slack against a floor of 9 is 1.22× of traffic, and a fused kernel gives back bandwidth: mean-field's streams eight buffers at once at 192.9 GB/s where the sequence it replaced walked three or four each at 240.8 GB/s. At that discount a 2.13× traffic cut is still 1.76× of time and a 1.22× cut is nothing, so a pass reduction quoted without a timing would have reported that arm as a 1.22× improvement.

Folding moves the amortisation curve rather than bending it: mean-field's parameter-sized share after folding is 0.68 of its share before at $B = 1$ and 0.60 at $B = 128$. Batching moves an arm along the curve, folding moves the curve down, and folding therefore returns most where amortisation has not arrived.

Arm	B	$O(P)$ ms before	$O(P)$ ms after	×	ms/step before	ms/step after	$O(P)$ % before	$O(P)$ % after	share ratio
BBB	1	5.92	3.36	1.76	12.90	10.71	45.9	31.4	0.68
BBB	2	5.90	3.38	1.74	14.25	11.99	41.4	28.2	0.68
BBB	4	5.88	3.44	1.71	17.48	15.23	33.7	22.6	0.67
BBB	8	5.87	3.46	1.70	26.36	24.18	22.3	14.3	0.64
BBB	16	5.86	3.48	1.68	45.07	43.18	13.0	8.1	0.62
BBB	32	5.86	3.50	1.67	81.03	79.27	7.2	4.4	0.61
BBB	64	5.86	3.48	1.68	150.46	149.81	3.9	2.3	0.60
BBB	128	5.86	3.54	1.65	293.16	295.16	2.0	1.2	0.60
Vadam	1	2.05	1.98	1.04	9.12	9.16	22.4	21.6	0.96
Vadam	2	2.05	1.99	1.03	10.44	10.42	19.6	19.1	0.98
Vadam	4	2.03	2.03	1.00	13.73	13.77	14.8	14.7	0.99
Vadam	8	2.03	2.04	1.00	22.61	22.69	9.0	9.0	1.00
Vadam	16	2.03	2.05	0.99	41.40	41.68	4.9	4.9	1.01
Vadam	32	2.02	2.06	0.98	76.83	77.78	2.6	2.6	1.01
Vadam	64	2.03	2.05	0.99	148.36	148.59	1.4	1.4	1.01
Vadam	128	2.02	2.07	0.98	293.82	294.54	0.7	0.7	1.02
LowR	1	39.94	34.48	1.16	48.26	42.66	82.7	80.8	0.98
LowR	2	40.03	34.77	1.15	49.61	44.20	80.7	78.7	0.97
LowR	4	40.34	35.18	1.15	52.84	47.49	76.3	74.1	0.97
LowR	8	40.47	35.45	1.14	62.11	56.99	65.2	62.2	0.95
LowR	16	40.37	35.28	1.14	81.06	76.04	49.8	46.4	0.93
LowR	32	40.42	35.57	1.14	117.51	113.13	34.4	31.4	0.91
LowR	64	40.11	35.06	1.14	187.41	182.67	21.4	19.2	0.90
LowR	128	40.43	35.21	1.15	334.62	329.50	12.1	10.7	0.88
Ens	1	1.49	1.50	1.00	8.56	8.45	17.4	17.7	1.01
Ens	2	1.49	1.49	1.00	9.95	9.83	14.9	15.1	1.01
Ens	4	1.49	1.48	1.00	13.33	13.18	11.1	11.2	1.01
Ens	8	1.48	1.48	1.00	22.37	22.16	6.6	6.7	1.01
Ens	16	1.47	1.47	1.00	41.50	41.20	3.6	3.6	1.01
Ens	32	1.48	1.47	1.00	78.24	77.70	1.9	1.9	1.00
Ens	64	1.47	1.47	1.00	148.24	148.82	1.0	1.0	1.00
Ens	128	1.47	1.48	0.99	295.64	292.52	0.5	0.5	1.02

Table 4: Parameter-sized work before and after folding, where both sweeps measured the point; the last column is the share after over the share before. The ensemble's code is identical in both, so its rows are the drift.

6 Correctness, and what the cost buys

Folding is a claim about implementation, so it needs a gate a wrong fold would fail. Evaluation at $S = 32$ on validation and test is byte-identical before and after on all four arms, and is the weakest of the three checks run: it draws with the unfused sample kernels, so a broken training path passes it. Running each fused kernel and the sequence it replaces on identical inputs leaves mean-field and Vadam bit-identical on every output and the low-rank arm within 1.00 ULP on A and its two moments, and rebuilding with multiply-add contraction disabled makes all three bit-identical, which puts the residual in contraction rather than in the arithmetic. And 100 steps of normal-mode training against the pre-folding source reproduce every printed step line on all four arms, including both summary lines of the control.

What the cost buys is read from the 100,000-step checkpoints, on both splits with neither preferred. The ensemble has the best mixture NLPD and the worst ECE by close to a factor of two and Vadam holds the opposite corner (Table 5); mean-field and the low-rank arm sit between them, separated by less than the two splits disagree about them. The ensemble also separates in- from off-distribution input by more than twice the margin of any variational arm (Table 7), whose epistemic terms barely move on uniform-random token ids.

A larger epistemic term is not a better posterior. Vadam's in-distribution epistemic is more

Arm	Split	S	NLPD (single draw)	NLPD (mixture)	ECE
BBB	val	32	1.5642	1.5582	0.0240
BBB	test	32	1.6611	1.6515	0.0169
Vadam	val	32	1.7119	1.6764	0.0168
Vadam	test	32	1.7748	1.7411	0.0136
LowR	val	32	1.5982	1.5819	0.0207
LowR	test	32	1.6806	1.6643	0.0211
Ens	val	5	1.5255	1.4708	0.0490
Ens	test	5	1.6251	1.5672	0.0379

Table 5: Predictive quality, in nats, over 16,384 positions. "Single draw" is one posterior sample, or member 0 for the ensemble: comparable within an arm, not across them.

Arm	Split	S	Total	Aleatoric	Epistemic (ID)
BBB	val	32	1.602	1.591	0.0107
BBB	test	32	1.660	1.649	0.0103
Vadam	val	32	1.705	1.668	0.0372
Vadam	test	32	1.737	1.701	0.0358
LowR	val	32	1.621	1.605	0.0165
LowR	test	32	1.673	1.657	0.0164
Ens	val	5	1.628	1.561	0.0672
Ens	test	5	1.690	1.622	0.0674

Table 6: In-distribution decomposition, in nats, by Equation 2. The worst residual against the asserted identity behind any number here is 4.4×10^{-15}.

than three times mean-field's (Table 6) while its separation ratio sits with the other variational arms, because its spread is the prior's: Equation 1 weighs the data's contribution to the precision, Ns, against $\lambda = 2500$, so the per-token RMS gradient would have to reach $\sqrt{\lambda/N} = 5.27 \times 10^{-3}$ where the median observed value is 2.71×10^{-5}.[1] Noise that does not shrink as the mean fits is also why that arm fails the one-batch memorisation gate. The knob would be λ, and choosing it by anything other than the initialisation is a modelling decision this comparison does not make.

Every uncertainty number above carries its S, because the estimator is Monte Carlo over S draws and biased low until it converges (Figure 1). Mean-field is flat by $S = 32$ and confirmed flat to $S = 128$, though its ECE rises at every doubling over that range while its mixture NLPD improves; the low-rank arm gains a few percent of epistemic between $S = 32$ and $S = 64$, so whether the convention moves to $S = 64$ is open.

7 What determines the cost

No arm here is limited by the arithmetic of its posterior. Every parameter-sized region in Table 1 runs within a few percent of every other one per buffer walked, so a strategy's per-step training cost on this card is its touch count times 44.2 MB divided by the bandwidth its kernels achieve: about

[1] Read from the trained checkpoint by `dev/vadam_posterior.py`, the one measurement here taken from a checkpoint rather than a logged run.

Arm	S	Epistemic (OOD)	OOD/ID (val)	OOD/ID (test)
BBB	32	0.0119	1.112	1.155
Vadam	32	0.0449	1.207	1.254
LowR	32	0.0208	1.261	1.268
Ens	5	0.1733	2.579	2.571

Table 7: Epistemic uncertainty on the off-distribution stream, in nats, stated once per arm: the stream is one fixed synthetic sequence, so only the ratio's denominator is split-dependent.

240 GB/s for a sequence walking three or four buffers each, about 190 for one kernel holding eight. That estimate is available from a method's definition, by counting the buffers it keeps and the reads and writes a step makes of each, less the eight touches a deterministic AdamW step was paying anyway.

The same count says which strategies a larger batch cannot rescue, since amortisation followed from the numerator being fixed, and it is fixed because the state is proportional to the parameter count and nothing else. A covariance factor of rank r multiplies that term by r, which is measured here: the low-rank arm still pays 11.65% of a step at $B = 128$. A per-example perturbation of the kind Wen et al. [25] move into the activations puts the sampling cost in the $O(B \cdot T)$ term instead, where it grows with the batch as the network does and its share does not fall; that follows from the mechanism and was not measured here. A covariance fitted after training [21, 3] or collected from the trajectory [16] moves the traffic out of the step, and MC dropout [4] never allocates any: all three pay at evaluation.

Where folding stops paying is also a count. Two of the three Bayesian arms reach the floor their state implies, and the gap between the traffic removed and the time recovered is bandwidth a wider kernel gives up, which grows with the live state it holds. The low-rank arm's remaining region is not a bandwidth problem at all: its KL setup blocks on host round-trips and small per-layer cuBLAS calls. Sweeping T as well as B would separate a dependence on tokens per step from one on batch size, and like everything above it runs from the checkpoints and logs already in the repository.

8 Limitations

The scale bounds the milliseconds, not the counts. 11,061,531 parameters on a 6 GB card, picked so all four arms fit at once. Touch counts do not depend on P, and neither does the flat numerator of Section 5.1, but every millisecond and every GB/s here belongs to this card. Where the network becomes token-bound does depend on the model: a larger one saturates the card at a smaller B and reaches the $1/B$ regime sooner. The vocabulary of 27 keeps the token embedding negligible, so nothing covers a posterior over a large embedding table.

The measurement is bounded by the card and by one axis. The card throttles, so the two sweeps are comparable only to about 1%: with per-point cooldown the later one still started a few degrees warmer and the same region comes out a few percent slower for it. That is what the unchanged control arm is for, and no claim in Section 5.3 rests on a difference smaller than the drift measured on it. The low-rank arm at $B = 256$ allocated with 248 MB of 6144 MB free and then ran at about four times what its own curve predicts, because past a couple of hundred megabytes of headroom the driver backs allocations with host memory and the step time stops being a property of the kernels; on WSL2 `cudaMalloc` succeeds in that state, so the guard is a headroom floor and

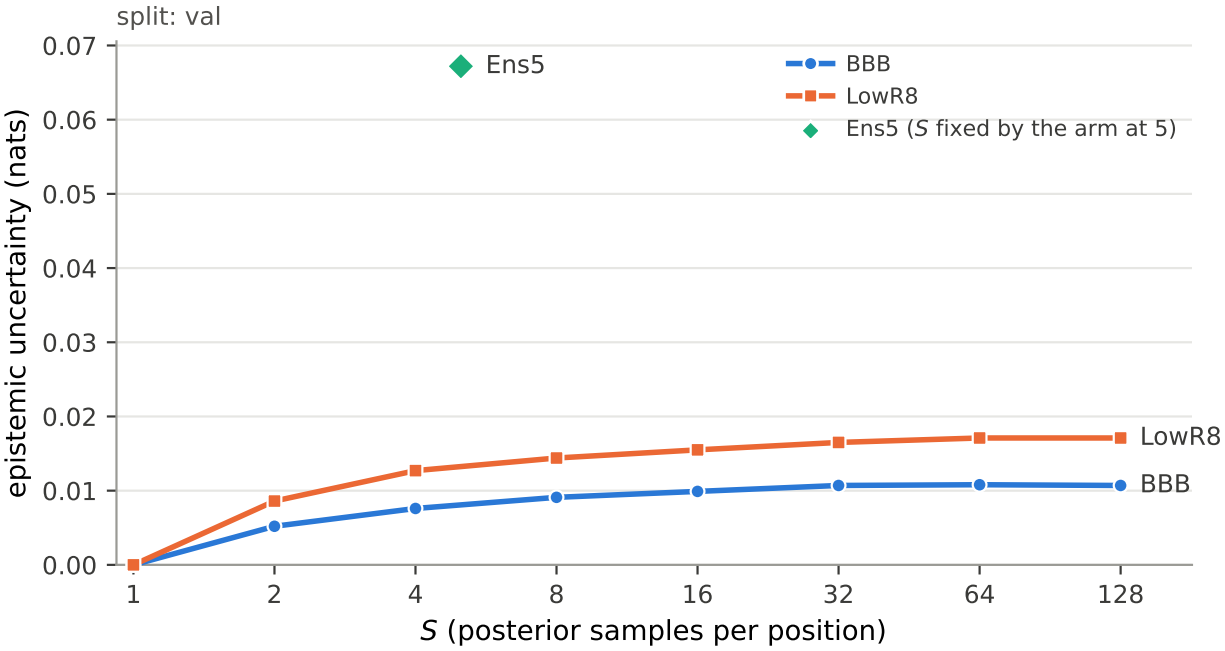


Figure 1: Epistemic uncertainty against posterior samples S, validation split. The foot at $S = 1$ is exactly zero by construction, and the ensemble is a marked point at the number of members it was paid to train.

that arm's sweep ends at $B = 128$. The sweep also varies B at $T = 64$ throughout, so what is measured is a dependence on batch size and not on tokens per step. One training run per arm, no seed sweep, no error bars, and three backward kernels accumulate with `atomicAdd`, so training is not bit-reproducible in principle; normal-mode runs at this configuration do reproduce, which is what the trajectory check of Section 6 uses, and one-batch overfit runs do not.

The quality comparison is confounded and unconverged. 100,000 steps at 256 tokens per step is 28% of one epoch, so every quality number is what a method does at a fixed step budget. Loss tracks the injected noise scale monotonically across arms, so an arm that samples less aggressively looks better on predictive metrics for a reason unrelated to its posterior, and the off-distribution stream is uniform-random token ids rather than shifted text, which supports no comparison with the shift literature [19]. The low-rank arm is Bayesian over 96% of the model and the ensemble evaluates at $S = E = 5$, so a mixture metric read across arms compares different coverage and different sample counts.

References

[1] Charles Blundell, Julien Cornebise, Koray Kavukcuoglu, and Daan Wierstra. Weight uncertainty in neural network. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37 of *Proceedings of Machine Learning Research*, pages 1613–1622. PMLR, 2015. URL https://proceedings.mlr.press/v37/blundell15.html.

[2] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast

and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html.

[3] Erik A. Daxberger, Agustinus Kristiadi, Alexander Immer, Runa Eschenhagen, Matthias Bauer, and Philipp Hennig. Laplace redux — effortless Bayesian deep learning. In *Advances in Neural Information Processing Systems 34 (NeurIPS)*, pages 20089–20103, 2021. URL https://proceedings.neurips.cc/paper/2021/hash/a7c9585703d275249f30a088cebba0ad-Abstract.html.

[4] Yarin Gal and Zoubin Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, volume 48 of *JMLR Workshop and Conference Proceedings*, pages 1050–1059. JMLR.org, 2016. URL http://proceedings.mlr.press/v48/gal16.html.

[5] Alex Graves. Practical variational inference for neural networks. In *Advances in Neural Information Processing Systems 24 (NIPS)*, pages 2348–2356, 2011. URL https://proceedings.neurips.cc/paper/2011/hash/7eb3c8be3d411e8ebfab08eba5f49632-Abstract.html.

[6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR, 2017. URL https://proceedings.mlr.press/v70/guo17a.html.

[7] Geoffrey E. Hinton and Drew van Camp. Keeping the neural networks simple by minimizing the description length of the weights. In *Proceedings of the Sixth Annual ACM Conference on Computational Learning Theory (COLT)*, pages 5–13. ACM, 1993. doi: 10.1145/168304.168306.

[8] Andrej Karpathy. nanoGPT. https://github.com/karpathy/nanoGPT, 2022.

[9] Andrej Karpathy. llm.c: LLM training in simple, raw C/CUDA. https://github.com/karpathy/llm.c, 2024.

[10] Alex Kendall and Yarin Gal. What uncertainties do we need in Bayesian deep learning for computer vision? In *Advances in Neural Information Processing Systems 30 (NIPS)*, pages 5574–5584, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/2650d6089a6d640c5e85b2b88265dc2b-Abstract.html.

[11] Mohammad Emtiyaz Khan, Didrik Nielsen, Voot Tangkaratt, Wu Lin, Yarin Gal, and Akash Srivastava. Fast and scalable Bayesian deep learning by weight-perturbation in Adam. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 2611–2620. PMLR, 2018. URL https://proceedings.mlr.press/v80/khan18a.html.

[12] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, 2015. URL http://arxiv.org/abs/1412.6980.

[13] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *2nd International Conference on Learning Representations (ICLR)*, 2014. URL http://arxiv.org/abs/1312.6114.

[14] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems 30 (NIPS)*, pages 6402–6413, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/9ef2ed4b7fd2c810847ffa5fa85bce38-Abstract.html.

[15] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *7th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2019. URL https://openreview.net/forum?id=Bkg6RiCqY7.

[16] Wesley J. Maddox, Pavel Izmailov, Timur Garipov, Dmitry P. Vetrov, and Andrew Gordon Wilson. A simple baseline for Bayesian uncertainty in deep learning. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 13132–13143, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/118921efba23fc329e6560b27861f0c2-Abstract.html.

[17] Matt Mahoney. About the test data. http://mattmahoney.net/dc/textdata.html, 2011. Large Text Compression Benchmark; source of `enwik8` and `text8`.

[18] Kazuki Osawa, Siddharth Swaroop, Mohammad Emtiyaz Khan, Anirudh Jain, Runa Eschenhagen, Richard E. Turner, and Rio Yokota. Practical deep learning with Bayesian principles. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 4289–4301, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/b53477c2821c1bf0da5d40e57b870d35-Abstract.html.

[19] Yaniv Ovadia, Emily Fertig, Jie Ren, Zachary Nado, D. Sculley, Sebastian Nowozin, Joshua V. Dillon, Balaji Lakshminarayanan, and Jasper Snoek. Can you trust your model's uncertainty? evaluating predictive uncertainty under dataset shift. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 13969–13980, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/8558cb408c1d76621371888657d2eb1d-Abstract.html.

[20] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

[21] Hippolyt Ritter, Aleksandar Botev, and David Barber. A scalable Laplace approximation for neural networks. In *6th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2018. URL https://openreview.net/forum?id=Skdvd2xAZ.

[22] John K. Salmon, Mark A. Moraes, Ron O. Dror, and David E. Shaw. Parallel random numbers: As easy as 1, 2, 3. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, pages 16:1–16:12. ACM, 2011. doi: 10.1145/2063384.2063405.

[23] Yuesong Shen, Nico Daheim, Bai Cong, Peter Nickl, Gian Maria Marconi, Clement Bazan, Rio Yokota, Iryna Gurevych, Daniel Cremers, Mohammad Emtiyaz Khan, and Thomas Möllenhoff. Variational learning is effective for large deep networks. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 44665–44686. PMLR, 2024. URL https://proceedings.mlr.press/v235/shen24b.html.

[24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NIPS)*, pages 5998–6008, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

[25] Yeming Wen, Paul Vicol, Jimmy Ba, Dustin Tran, and Roger B. Grosse. Flipout: Efficient pseudo-independent weight perturbations on mini-batches. In *6th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2018. URL https://openreview.net/forum?id=rJNpifWAb.